

## ANONYMOUS AUTHOR(S)

**Additional Key Words and Phrases:** LLM agents, Model Context Protocol, tool surface, least privilege, configuration analysis, information flow, prompt injection, empirical study

### ACM Reference Format:

Anonymous Author(s). 2026. INTERLOCK. 1, 1 (September 2026), 14 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

## 1 Introduction

에이전트가 외부 도구를 받아오는 표준은 MCP(Model Context Protocol)다 [9]. 사용자는 설정 파일에 서버를 적고, 서버마다 그것을 실행할 명령, 서버가 받을 자격증명, 그리고 (일부 서버는) 어떤 도구를 광고할지 정하는 플래그를 함께 적는다. 그러면 호스트가 서버를 띄우고 도구 목록을 받아, 각 도구의 이름과 설명, 스키마를 모델에게 보여준다. 이렇게 한 세션에 모인 도구 전부를 그 배포의 *tool surface*, 곧 모델이 부를 수 있는 도구의 집합이라 부른다. 이 방식은 이미 굳어졌다. 실제 애플리케이션 1,723개 중 85.2%가 서버 목록을 파일로 두는 반면, 도구 호출을 멈추고 사용자 승인을 받는 차단형 게이트를 두는 것은 37.2%뿐이다 [20]. 서버 한 줄을 적으면, 대개 그 서버의 도구는 모델이 부르기만 하면 실행된다.

이 충위를 보호하는 연구는 세 갈래다. 개별 검사는 도구 하나 또는 서버 하나를 놓고 위험한 지를 판정해, 광고된 설명의 숨은 지시와 이름 가로채기, 서버 구현 안의 결함, 실행 중의 이상 동작을 찾아낸다 [13, 28–30, 34]. 조합 검사는 도구의 능력 라벨을 받아, 신뢰할 수 없는 콘텐츠와 비공개 데이터, 그리고 공격자가 읽을 수 있는 목적지가 한 세션에 함께 모이는 조합을 위험으로 지목한다 [2, 24, 31]. 권한 제한은 사용자의 작업에서 도구 권한 정책을 만들어 호출마다 집행한다 [6, 7, 26].

실제 배포에서는 그 전체가 성립하지 않는다. 그림 1은 우리가 다루는 상황으로 MCP를 이용하여 자기 저장소의 열린 이슈를 정리하는 과정을 보여준다. 먼저 ①에서 개발자가 코딩 에이전트에 GitHub MCP 서버 하나를 붙인다. ②에서 호스트가 그 서버에 도구 목록을 요청하는데, 설정에 적힌 것은 서버 한 줄이지만 돌아오는 도구는 우리가 계측한 버전 기준 26개이고 모두 모델이 부를 수 있다. 이 가운데 셋이 모이면 유출 경로가 된다. ③의 `list_issues`는 누구나 쓸 수 있는 공개 이슈를 컨텍스트로 들여오고, ④의 `get_file_contents`는 같은 토큰으로 비공개 저장소를 읽으며, ⑤의 `add_issue_comment`는 그 내용을 공격자가 읽을 수 있는 공개 이슈에 쓴다. 공격자가 공개 이슈에 지시문을 심어 두면, 에이전트는 그 이슈를 읽다가 지시를 따른다. 이런 경로는 손대지 않은 GitHub MCP 서버 하나만 연결한 상태에서 실제로 시연되었다 [21]. 그런데 개별 검사는 ③과 ④, ⑤를 하나씩 보기 때문에 여기서 아무것도 지목하지 않는다 [13, 28, 29]. 조합 검사는 셋이 함께 쓰일 때를 보므로 이 경로를 정확히 지목할 수 있다 [2, 24]. 그러나 거기서 멈춘다. 이 배포가 위험하다고 판정할 뿐 무엇을 꺼야 하는지는 말하지 않으므로, 사용자에게 남는 선택은 경고를 무시하거나 서버를 통째로 지우는 것뿐이다. 대책이 될 수 있는 것은 권한 제한뿐이다. 작업에서 정책을 만들어 ④를 끄는 데까지 간다 [6, 7, 26]. 다만 그러려면 ④가 비공개 데이터에 닿는다는 것을 먼저 알아야 하는데, 그 정보는 제공자가 쓴 이름과 설명에 없고 기계가 읽을 형태로 어디에도

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/9-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

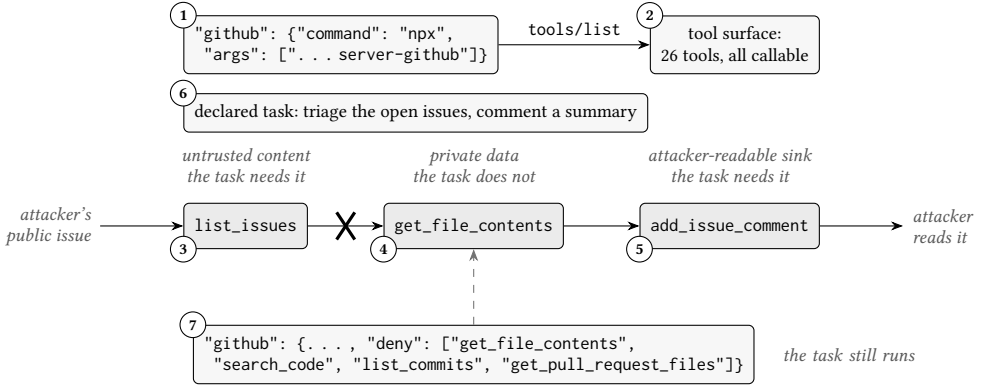


Fig. 1. Example of a leak from one MCP server, the task the server was configured for, and the configuration change INTERLOCK returns.

선언되어 있지 않다. 이 예시가 드러내는 것은 일반적인 한계다. 권한 제한은 도구가 무엇에 닿는지를 제공자가 쓴 글에서 읽고, 그 글이 실제 구현과 맞는지는 아무도 확인하지 않는다. 그래서 글이 어긋나면 정책도 어긋나는데, 어긋났다는 사실이 드러나지 않는다. 비공개 데이터에 닿는 도구를 닿지 않는다고 믿고 열어 둔 채로 최소 권한을 집행했다고 보고하게 되며, 틀린 정책과 맞는 정책이 구별되지 않는다. 더구나 그 글은 우리 위협 모델에서 공격자가 쓸 수 있는 것이어서, 설명을 고치면 정책이 따라 바뀐다. 게다가 안다고 해도 끝이 아니다. 이 경로는 세 도구 가운데 어느 하나만 꺼도 끊어지므로, 무엇을 끝지는 ⑥의 선언된 작업이 정한다. 그 작업이 열린 이슈를 정리해 요약은 코멘트로 남기는 것이라면 ③과 ⑤는 작업이 쓰는 도구여서 끝 수 없고, ④만 끝 수 있다. 무엇을 끝지는 위협의 성질이 아니라 배포가 선언한 작업이 정하는 것이다. 그래서 도구를 하나씩 끄는 수단이 있고 정책을 만들 줄 아는 연구가 있어도, ⑦과 같은 정밀한 변경을 계산해 주는 것은 없다.

이 논문에서 우리는 INTERLOCK을 제시한다. 도구가 무엇에 닿는지를 제공자가 쓴 글이 아니라 서버의 소스 코드에서 회수하되 모든 주장을 기계적으로 재검사하고, 그렇게 얻은 효과 요약이 실제로 쓸 수 있는 정보임을 설정 변경을 계산해 보이는 접근이다. *Summarize*는 신뢰도 순으로 정렬한 증거에서 도구마다 어떤 효과에 닿는지를 판정하고, 판정마다 그것을 뒷받침하는 코드 위치를 남긴다. 이 요약은 그 자체로 앞 문단의 빈자리를 채우는 산출물이며, 런타임 최소 권한 시스템이 사람 손으로 채우던 라벨을 대신할 수 있다. 우리는 이어서 그 요약이 설정 수준에서 실행 가능함을 보인다. *Decide*는 설정이 지정한 값 수준 조건을 적용한 뒤 요약 위에서 흐름 정책을 평가해 금지된 흐름마다 그림의 ③-⑤처럼 그것을 만드는 도구를 짚는다. *Repair*는 호스트와 서버가 이미 가진 수단 — 도구 거부 항목, 서버 모드와 toolset 플래그, 인자 축소, 마지막 수단인 서버 제거 — 위에서 선언된 작업을 지키는 최소 비용 patch를 만들고, 그 결과가 ⑦의 설정 diff다. 집행에 호스트를 고칠 필요는 없다.

핵심 난점은 그 정보가 선언되어 있지 않다는 것이다. 효과를 코드에서 읽어내야 하는데, 두 길 모두 쉽지 않다. 공개 패키지를 전부 정적 분석으로 따라가면 규모가 맞지 않고, 언어 모델에 읽히면 그 판정을 무엇으로 믿느냐가 남는다. 우리는 신뢰도 순위 요약과 결정적 재검사를 묶어 이를 푼다. 모델이 서버의 소스를 읽고 도구별 효과를 판정하되 주장마다 file:line 증거를 달게 하고, 결정적 검사기가 가져온 소스에서 그 줄을 다시 읽어 확인되지 않는 주장을 걸러낸다. 확인되지 않은 주장은 버리지 않고 “미확인”으로 표시해 라벨을 지키며, 모델이 판정하지 않은 도구에는 가장 보수적인 값을 준다. 문서만 근거로 한 “무해” 판정은 무해로 세지 않는다. 문서는 효과가 없다는 것을 증명하지 못하기 때문이다. 그래서 출력은 그럴듯한 요약이 아니라, 독자가 코드 한 줄까지 되짚을 수 있는 감사 가능한 기록이 된다.

**Contributions.** We summarize our contributions as follows:

- **Observation:** 우리는 에이전트 최소 권한 기법들이 공통으로 전제하는 것, 곧 도구마다 무엇에 닿는지가 이미 알려져 있다는 전제를 정리하고, 그 전제를 세 시스템이 스스로 가정이나 한계로 적고 있음을 보인다. 또한 그 전제를 실제 서버에 대해 채우려는 유일한 선행 도구가 사람들이 실제로 선언하는 서버 대부분을 차지하는 생태계에서 작동하지 않음을 보인다.
- **New technique:** 우리는 공격자가 쓸 수 있는 산문을 결코 신뢰하지 않고 서버의 소스 코드에서 도구별 효과를 회수하는 절차를 제시한다. 모델의 모든 판정에 `file:line` 증거를 요구하고 결정적 검사기로 그 구간을 다시 읽어 확인되지 않은 주장을 표시하며, 미판정 도구에는 보수적인 값을 주고 문서만으로는 무해를 인정하지 않는다. 그 위에서 설정에 대한 최소 권한 `tool surface` 합성을 정식화하고, `witness` 유도 수선과 제공자가 `surface`를 바꿀 때의 재판정을 제시한다.
- **Evaluation:** 우리는 이 절차를 구현해 공개 설정에서 가장 자주 선언되는 서버 34개와 그 도구 433개에 적용하고, 모델 판정 가운데 결정적 재검사를 통과하는 비율(전체 72.5%, 구현 코드가 배포되는 경우 90.7%)을 보고한다. 또한 공개된 패키지를 읽는 것만으로는 효과를 확정할 수 없는 세 가지 구조적 한계를 식별한다. 도구와 코드에서 도출한 효과 요약을 공개한다.

## 2 Preliminaries

### 2.1 Tool surfaces and who decides them

호스트 애플리케이션 – IDE 어시스턴트, 데스크톱 에이전트, 코딩 에이전트 – 은 설정 파일에 명시된 각 Model Context Protocol 서버를 실행하고 도구 목록을 요청한 뒤, 반환된 이름과 설명, JSON 스키마를 모델의 컨텍스트에 넣는다. 서버는 명령줄로 실행되는 로컬 프로세스일 수도, 컨테이너일 수도, 원격 엔드포인트일 수도 있으며, 스킵 번들이 서버를 추가로 기여하기도 한다. 한 세션에서 광고된 도구들의 합집합을 우리는 그 배포의 *tool surface*라 부른다. 이 구조의 세 가지 성질이 이후 논의를 이끈다. *surface*는 개별 제공자가 아니라 사용자의 설정이 결정한다. *surface* 위의 모든 도구는 호출 가능하다. 도구 선택은 광고된 텍스트를 읽은 모델이 하기 때문이다. 그리고 *surface*는 고정되어 있지 않다. 호스트는 대개 버전을 고정하지 않은 패키지 지정으로 서버를 실행하므로, 도구 뒤의 코드는 사용자가 아무 행동을 하지 않아도 세션 사이에 바뀔 수 있다.

### 2.2 Threat model

우리는 에이전트가 자신이 읽은 콘텐츠 안의 지시를 따른다고 가정한다. 이는 *lethal trifecta* 프레임 [31]과 *toxic-flow* 분석 [2]이 전제하는 가정이며, 이 층위에 대해 공개된 공격들이 실제로 악용하는 가정이다 [16, 35, 36]. 공격자는 임의의 웹 호스트와 콘텐츠를 통제하고, 여러 사람이 쓸 수 있거나 제3자가 제공하는 출처의 콘텐츠 – 공개 이슈와 댓글, 저장소 내용, 커뮤니티 문서와 레지스트리, 검색 결과 – 도 통제한다. 공격자는 사용자의 로컬 파일이나 사용자 계정에 담긴 데이터를 통제하지 못하며, 설치된 서버를 수정하지도 못한다. 공격자는 자신이 통제하는 호스트와 공개된 모든 것을 읽을 수 있다. 이 가정 아래에서, 어떤 배포의 *surface*가 한 세션 안에서 비공개 데이터 출처, 공격자가 통제하는 콘텐츠 출처, 공격자가 읽을 수 있는 목적지에 모두 닿을 때 그 배포는 위험하다. 우리는 그런 *surface*를 *trifecta-complete*라 부르고, 자기 도구만으로 충분한 제공자를 *capability-bundling*이라 부른다.

### 2.3 Deciding what to switch off

무엇을 꺼야 하는지에 관여하는 두 갈래, 곧 조합 검사와 권한 제한이 우리의 직접적인 비교 대상이다. 여기에 그 둘이 전제하는 라벨을 직접 뽑으려는 시도를 더해 방법론을 차례로 본다.

**Progent.** Progent는 권한을 도구 이름과 인자에 대한 기호 규칙의 집합으로 표현한다 [26]. 사용자의 질의가 주어지면 LLM이 작업 수행에 필요한 호출만 허용하고 불필요한 호출은 막는 초기

Table 1. Where each system obtains what a tool does, and what it returns.

| System             | Per-tool effects come from        | Evaluated on     | Output                  |
|--------------------|-----------------------------------|------------------|-------------------------|
| Progent [26]       | provider’s names and descriptions | simulated suites | per-call decision       |
| CaMeL [7]          | hand-written annotations          | simulated suites | per-call decision       |
| Fides [6]          | assumed correct via wrappers      | simulated suites | per-call decision       |
| mcp-sec-audit [12] | keyword and regex rulebook        | 45 servers       | risk score and advice   |
| Set-level [2, 24]  | taken as input, unstated          | 32 tools         | block or revoke session |
| INTERLOCK          | server source code, re-checked    | 34 servers       | per-tool effect summary |

정책을 자동으로 생성하며, 실행 중 새 정보가 들어오면 정책을 갱신한다. 모든 도구 호출은 결정적 절차로 정책과 대조되고, 이 집행기는 LangChain이나 OpenAI Agents SDK 같은 에이전트 프레임워크 안에 통합된다. 정책 생성기가 입력으로 받는 것은 최초 질의와 사용 가능한 도구 집합이다.

**CaMeL.** CaMeL은 신뢰된 질의에서 제어 흐름과 데이터 흐름을 먼저 뽑아낸다 [7]. 권한 있는 LLM이 질의를 제한된 프로그램으로 옮기고, 격리된 LLM이 신뢰할 수 없는 데이터를 해석하며, 전용 인터프리터가 값마다 능력(capability)을 추적해 도구가 호출되는 시점에 정책을 집행한다. 이로써 신뢰할 수 없는 데이터가 프로그램의 흐름에 영향을 주지 못한다. 정책은 임의의 로직을 담을 수 있도록 DSL 대신 Python 코드로 작성되며, 현재 구현에서는 CaMeL 엔진이 정의한다. 도구 또한 자기 기능에 따라 능력을 부여하는데, 어떤 주석을 어느 입도로 붙일지는 사용 사례마다 정해진다.

**Fides.** Fides는 기밀성과 무결성 라벨을 실행 전반에 전파하는 플래너다 [6]. 라벨은 대화 이력의 메시지, 도구 인자, 도구 결과, 데이터저장소 변수에 붙고, 도구 결과 트리의 각 노드에 메타데이터 필드로 저장된다. 플래너는 이 라벨을 보고 다음 단계를 정하며, 필요하면 정보를 선택적으로 감춘다. 이 전파가 성립하려면 모든 도구가 자기 출력에 올바른 라벨을 붙여야 하는데, Fides는 신뢰할 수 없는 모든 도구에 신뢰된 wrapper를 둬으로써 이를 확보한다.

**mcp-sec-audit.** 이 도구는 위 세 시스템이 전제하는 도구별 능력을 직접 뽑으려는 시도다 [12]. TOML 규칙집에 능력군과 그 지표가 되는 키워드 및 정규식, 심각도 가중치를 적어 두고, 정적 파이프라인이 Python 소스와 MCP 도구 메타데이터 JSON을 훑어 file\_read, file\_write, network\_outbound, command\_exec, env\_access 같은 능력을 탐지한다. 선택적으로 Docker 샌드박스에서 도구를 실행하고 eBPF로 커널 이벤트를 수집하는 동적 파이프라인을 함께 돌린다. 산출물은 탐지된 능력의 신뢰도를 가중합한 위험 점수와, 그 능력에 대응하는 강화 권고다.

**조합 검사.** lethal trifecta 프레이밍은 비공개 데이터와 신뢰할 수 없는 콘텐츠, 그리고 외부 전송 – 곧 공격자가 읽을 수 있는 목적지 – 가 한 세션에 모이는 것을 위험으로 명명한다 [31]. toxic-flow 분석은 도구 흐름, 곧 가능한 도구 사용 순서를 신뢰 수준과 민감도, 유출 지점 여부 같은 속성과 함께 모델링한다 [2]. 조합 정책은 안전하지 않은 사슬이 시작되기 전에 조합을 차단하고, 누적된 노출로 다음 호출이 위반이 되면 세션을 취소한다 [24]. 개별 검사와 달리 이들은 도구 하나가 아니라 조합을 판정하며, 그림 1의 유출 경로를 정확히 지목할 수 있다. 다만 이들은 도구의 능력 라벨을 입력으로 받는다. 라벨을 어디서 얻는지는 두 문헌 모두 명시하지 않으며, 평가 규모는 도구 32개에 정책 16개다. 서버 하나가 도구 26개를 광고하는 환경에서는 그 출처가 문제가 되지만, 도구 32개 규모에서는 사람이 붙여도 되는 일이다.

## 2.4 Limitation

표 1가 비교 대상을 정리한다. 앞의 세 시스템은 도구 단위로 권한을 좁히지만, 각 도구가 무엇에 닿는지를 스스로 확정하지는 않는다. Progent의 정책 생성기가 보는 것은 제공자가 쓴 이름과 설명과 스키마이고, CaMeL의 능력 주석은 사용 사례마다 사람이 정하며, Fides는 신뢰된 wrapper를 통해 모든 도구가 올바르게 라벨을 붙인다고 가정한다. 그런데 §2.2의 위협 모델에서 그 이름과

설명은 공격자가 쓸 수 있는 글이고, 프로토콜 명세 자신도 신뢰된 서버에서 온 것이 아닌 한 도구 annotation을 믿지 말라고 클라이언트에 요구한다. 사람이 손으로 붙이는 주석은 서버 수와 버전 수에 비례해 늘어나는데, 공개 벤치마크는 서버 515개의 버전 9,273개에 걸쳐 초기 도구의 54.6%가 수정되거나 폐기되었다고 보고한다 [15].

더구나 세 시스템의 평가는 실제 MCP 서버가 아니라 벤치마크 안에 정의된 가상 도구 모음 위에서 이루어진다. 그런 환경에서는 도구의 설명과 구현을 같은 저자가 같은 파일에 쓰므로, 도구가 무엇을 하는지 안다는 전제가 구조적으로 참이다. 다시 말해 전제가 깨지는 조건이 평가에 들어 있지 않다.

그 전제를 실제 서버에 대해 채워 주려는 시도는 우리가 아는 한 mcp-sec-audit 하나뿐이며, 저자들이 세 가지 한계를 스스로 보고한다. 첫째, 정적 분석기가 Python과 JSON 메타데이터만 지원해 JavaScript 서버 7개 중 0개를 탐지한다. 그런데 사람들이 실제로 선언하는 서버는 대부분 npm이다(표 2에서 npm 항목 22,010개, PyPI 3,156개). 둘째, 패턴 기반 탐지는 위험한 키워드와 우연히 일치하는 정상 코드에 오탐하고 난독화되거나 간접적으로 호출되는 경우를 놓친다. 셋째, 산출물이 위험 점수와 일반적인 권고여서, 저자들의 표현대로 “배포 가능한 설정을 생성하지 않는다”. 그리고 어느 경우에도 판정을 코드로 되짚는 장치는 없다.

그림 1의 설정에 이 한계들이 어떻게 드러나는지 보면 분명해진다. 개별 검사는 아무것도 지목하지 않는다. 세 도구 가운데 설명에 숨은 지시가 있거나 구현에 taint 결함이 있는 것이 없기 때문이다. 조합 검사는 유출 경로를 정확히 지목할 수 있지만 판정에서 멈춘다. 설정을 바꾸지도, 어느 도구를 꺼야 하는지 말하지도 않는다. 게다가 그 판정조차 ③이 신뢰할 수 없는 콘텐츠를 들여오고 ④가 비공개 데이터에 닿으며 ⑤가 공격자가 읽을 수 있는 곳에 쓴다는 것을 먼저 받아야 가능한데, 그 정보가 바로 아무도 만들지 않는 것이다. 행위까지 가는 것은 권한 제한뿐인데, 세 시스템 모두 같은 자리에서 멈춘다. Progent가 이 작업에 필요한 호출만 허용하는 정책을 쓰려면 ④가 비공개 저장소를 읽는다는 것을 알아야 하는데, 그것을 도구 이름과 설명에서 읽어야 한다. Fides가 라벨을 전파하려면 이 서버의 26개 도구에 신뢰된 wrapper가 있어야 하고, CaMeL이라면 그 26개에 능력 주석을 사람이 붙여야 한다.

그 정보가 주어졌다고 해도 문제가 남는다. 이 경로는 세 도구 가운데 어느 하나만 꺼도 끊어 지므로, 경로를 지목하는 것만으로는 무엇을 꺼야 할지가 정해지지 않는다. ⑥의 선언된 작업이 열린 이슈를 정리해 요약해 코멘트로 남기는 것이라면, ③은 이슈를 읽는 도구이고 ⑤는 요약을 남기는 도구여서 둘 다 끌 수 없다. 저장소 파일을 읽는 ④만이 끌 수 있는 도구다. 무엇을 끌지는 위험의 성질이 아니라 배포가 선언한 작업이 정하며, 이것은 탐지 문제가 아니라 작업 보존 제약 아래의 선택 문제다. 위 어느 시스템도 이 제약을 명시적으로 다루지 않는다. 끝으로 Progent와 CaMeL, Fides는 판정을 실행 시점에 내리므로 판정이 옳더라도 사용자에게 남는 산출물이 없다. 같은 설정을 내일 다시 열면 같은 판정을 처음부터 다시 만들어야 하고, 제공자가 도구를 추가해도 어제의 판정이 무효가 되었다는 사실은 어디에도 기록되지 않는다.

## 2.5 Goal

이 논문의 목표는 실제 MCP 서버의 도구마다 무엇에 닿는지를, 제공자가 쓴 글이 아니라 배포된 코드에서, 독자가 되짚을 수 있는 형태로 회수하는 것이다. 이 산출물은 위 세 시스템이 사람 손으로 채우거나 가정으로 두던 자리를 대신할 수 있고, 설정 시점에 직접 쓰일 수도 있다.

두 가지가 어렵다. 첫째, 공개된 패키지를 전부 정적 분석으로 따라가는 것은 규모가 맞지 않는다. 서버는 TypeScript와 Python, Go, Rust 등으로 작성되고, 번들이나 minify된 형태로, 또는 바이너리를 내려받는 실행기로 배포되며, 얇은 wrapper가 실제 구현을 의존성에 두기도 한다. mcp-sec-audit이 JavaScript에서 멈춘 지점이 바로 여기다. 둘째, 언어 모델에 코드를 읽히면 규모 문제는 사라지지만 그 판정을 무엇으로 믿을지가 남는다. 모델은 읽지 않은 것을 그럴듯하게 서술할 수 있고, 판정 대상이 바로 공격자가 쓸 수 있는 텍스트를 포함하고 있다. 따라서 목표는 판정을 얻는 것만이 아니라, 얻은 판정 가운데 코드로 뒷받침되는 부분과 그렇지 않은 부분을 기계적으로 가르는 것까지를 포함한다.

Table 2. Configuration corpus mined from public GitHub. Entries are server declarations; one configuration may declare several.

|                                              |                         |
|----------------------------------------------|-------------------------|
| Configurations (deduplicated, non-fork)      | 19,777                  |
| declaring a single server                    | 10,532                  |
| declaring two or more servers                | 9,302                   |
| Server entries by kind: npm                  | 22,010                  |
| remote endpoint                              | 14,250                  |
| other command                                | 7,564                   |
| local script                                 | 7,284                   |
| PyPI                                         | 3,156                   |
| container image                              | 1,507                   |
| Stdio entries launched without a version pin | 24,354 / 26,673 (91.3%) |

3 How Dangerous Are Real Configurations?

3.1 Study design

이 절의 코퍼스가 하는 첫 번째 일은 피험자 선정이다. 어떤 서버를 읽을 가치가 있는지를 저자의 판단이 아니라 공개 설정에 등장하는 빈도로 정한다. 그 위에서 우리는 앞 절에서 정의한 두 성질도 측정하지만, 그 비율은 이차적 증거이며 §5.4가 적는 표집 한계 안에서 읽어야 한다. 우리는 위에서 정의한 두 성질을, 사람들이 실제로 작성한 설정에서 측정한다. 현재 호스트들이 읽는 설정 형식 다섯 가지(.mcp.json, .cursor/mcp.json, .vscode/mcp.json, claude\_desktop\_config.json, 그리고 .codex/config.toml의 [mcp\_servers] 테이블)를 공개 GitHub에서 검색하고, 각 파일을 메모리에서 파싱해 서버의 식별자와 실행 형태만 보관한다. 자격증명은 저장하지 않는다. 검색은 85,065개의 파일을 보고했고, API의 질의당 상한 아래에서 22,552개를 받을 수 있었으며, 21,015개가 파싱되고, 포크와 저장소 내 중복 서버 집합을 제거한 뒤 19,777개가 남았다. 표 2가 코퍼스를 요약한다.

어떤 설정이 trifecta-complete인지 판단하려면 광고된 각 도구가 무엇에 닿는지 알아야 한다. 그래서 우리는 각 서버의 tools/list 핸드셰이크를 컨테이너 안에서 실행해 광고된 surface를 복원하고, 가장 자주 설정되는 서버 17개의 모든 도구에 대해 서버 자신의 소스 코드에서 효과를 도출하되 효과마다 파일과 줄 번호를 기록하며, 각 설정이 실제로 지정한 모드 플래그와 인자를 적용한다. 4절이 이 절차를 설명하며, 이는 INTERLOCK의 첫 단계다. 5절은 그 절차가 얼마나 신뢰할 만한지 보고한다. 이 절에서 필요한 것은 각 효과가 독자가 직접 확인할 수 있는 코드 한 줄로 추적된다는 사실뿐이다.

3.2 Results

분석 가능한 설정의 대부분이 trifecta-complete이다. 판정 대상 서버 17개로만 구성된 설정 880개 – 우리가 수집한 다중 서버 설정의 10분의 1 – 가운데 84.1%(740개)가 trifecta-complete이다. 코드로 확정하지 못한 효과를 양극단으로 밀어도 값은 84.0%에서 85.6% 사이로만 움직인다. 사용자 자신의 데이터베이스와 워크스페이스 안의 레코드도 공격자가 쓸 수 있다고 보는 두 번째 위협 모델 변형에서도 값은 84.1%로 변하지 않는데, 해당 서버들이 이미 다른 경로로 세 효과 모두에 닿기 때문이다.

그것들을 위험하게 만드는 것은 조합이 아니다. trifecta-complete인 740개 설정 전부에서 단일 서버가 이미 capability-bundling이다. trifecta-complete가 되기 위해 서버 두 개 이상이 필요한 설정의 수는 0이다. 브라우저와 fetch 서버가 file://, localhost, 인트라넷 URL에 닿아 얻는 비공개 데이터 효과를 제외해도 그 비율은 3.3% 이하에 머물고, 여기에 더해 문서화된 원격 코드 실행 도구와 모든 로컬 URL 효과까지 무시해야 19.1%에 이르는데, 그 시점에는 남은 사례 대부분이 한

Table 3. Servers that make a configuration trifecta-complete by themselves, with the number of analysed configurations in which each appears.

| Server                   | Configs | Why it alone suffices                                |
|--------------------------|---------|------------------------------------------------------|
| @playwright/mcp          | 359     | default tool runs agent code in the server process   |
| github-mcp-server        | 183     | private repos, third-party issues, public writes     |
| server-github (archived) | 172     | same, with no mode flags at all                      |
| chrome-devtools-mcp      | 139     | navigates to file:// and localhost, then exfiltrates |
| shadcn                   | 121     | fetches agent-chosen URLs, reads local files         |
| supabase                 | 81      | user data in, agent-supplied code out                |
| serena (default context) | 28      | host shell execution                                 |
| mcp-server-fetch         | 26      | no host restrictions on the URL                      |
| server-puppeteer         | 14      | agent-supplied launch options reach spawn            |

문서 서버의 자격증명 의존 효과에 기대고 있다. 17개 서버 중 9개가 단독으로 capability-bundling이며, 그중에는 수천 개 설정에 등장하는 브라우저 자동화, 코드 호스팅, 데이터베이스, 컴포넌트 레지스트리 서버가 있다(표 3).

존재하는 제어 수단은 쓰이지 않고, 주요 원인 서버에는 아예 없다. 코드 호스팅 서버는 읽기 전용 모드를 제공하지만 항목 189개 중 3개만 이를 설정하고 12개가 더 좁은 toolset을 선택한다. 데이터베이스 서버는 읽기 전용 모드를 제공하며 항목 116개 중 32개가 이를 설정한다. 코딩 에이전트 서버는 이름이 지정된 대부분의 context에서 셸 도구를 비활성화하는데, 항목 111개 중 84개가 그런 context를 선택한다. 채택률이 높은 유일한 사례다. 브라우저 자동화 서버에는 자기 프로세스 안에서 코드를 실행하는 도구를 제거하는 플래그가 없으며, archived 코드 호스팅 서버와 컴포넌트 레지스트리 서버, fetch 서버는 제한을 거는 플래그를 전혀 제공하지 않는다.

surface는 고정되어 있지 않다. 널리 설정되는 한 서버의 연속된 릴리스 64개 가운데 45개가 광고된 surface를 바꿨고 설명만 바뀐 것은 2개뿐이었다. 공개된 벤치마크는 서버 515개의 버전 9,273개에 걸쳐 초기 도구의 54.6%가 수정되거나 폐기되었다고 보고한다 [15]. 한편 stdio 항목의 91.3%는 버전을 명시하지 않으므로 도구 뒤의 코드는 사용자가 아무 행동을 하지 않아도 바뀐다. 해시 기반 pinning은 이 모든 변화에 반응한다. 우리 코퍼스를 복원한 버전 이력에 대해 재생하면 18개월 창에서 설정당 약 23건의 변경 경고가 발생한다.

레지스트리 규모 조사와의 대조. 서버 1,360개의 도구 12,230개를 대상으로 한 조사는 한 서버가 세 능력을 동시에 가질 수 있고 그 경우 서버 사이 협조 없이 공격을 완성할 수 있다고 보고하면서, 그런 서버를 2개로 센다 [36]. 우리의 비율이 훨씬 높은 이유는 두 가지다. 그 조사는 레지스트리에서 서버를 균등하게 표집하지만 우리는 사람들이 실제로 설치하는 빈도로 가중치를 준다. 또한 우리는 도구 메타데이터가 아니라 소스 코드에서 효과를 도출하므로, 광고된 도구에서 도달 가능한 호스트 코드 실행이나 브라우저·fetch 도구가 로컬 URL로 얻는 파일 내용처럼 메타데이터 수준 분류기가 보지 못하는 효과를 센다. 두 결과는 방향에서 일치하고 비율에서 갈리며, 어느 쪽이든 단일 서버가 충분하다는 결론은 같다.

### 3.3 What this rules out

이 수치들은 설계 공간을 세 가지 방식으로 제약한다. 이 기저율에서 경고는 정보를 주지 못한다. 집합 단위 trifecta 검사는 이 설정들의 84%에 경고를 띄우고, 사용자가 할 수 있는 일은 무시하는 것뿐이다. 실제로 어떤 벤더 검사기는 바로 이 점을, 즉 대부분의 에이전트가 세 특성을 모두 가지므로 각 구성요소를 개별적으로 분석한다고 밝힌다 [27]. 서버 제거는 지나치게 거칠다. 판정을 좌우하는 서버들이 사용자가 의도적으로 설치한 것들이기 때문이다. 그리고 조합 지향 방어가 함의하는 해법인 제공자 사이 흐름 차단은 740개 설정 전부를 여전히 trifecta-complete로 남긴다.

남는 길은 사용자가 유지하는 제공자 내부에서 surface를 좁히는 것이며, 그것이 INTERLOCK이 하는 일이다.

## 4 Approach

### 4.1 Overview

INTERLOCK은 설정, 흐름 정책, 배포가 선언한 작업을 입력으로 받아, 호스트와 서버가 이미 제공하는 제어 수단 위의 patch를 반환한다. 절차는 세 단계다. *Summarize*는 신뢰도 순으로 정렬한 증거에서 도구별 효과 요약에 도출한다. *Decide*는 설정이 유도한 surface에 대해 정책을 평가하고 각 금지 흐름을 증언하는 도구를 반환한다. *Repair*는 최소 비용 patch – 도구 단위 거부 항목, 서버 모드와 toolset 플래그, 인자 축소, 그리고 가장 비싼 선택지인 서버 제거 – 를 탐색해 금지된 흐름을 제거하면서 선언된 작업이 계속 동작하도록 한다. 이 논문은 *Summarize*를 구현·평가하며, *Decide*와 *Repair*는 여기서 명세하고 진행 중인 작업으로 남겨 두되, 두 단계에 관한 모든 진술은 그에 맞게 표시한다.

### 4.2 Summarize: trust-ranked effect summaries

이 단계는 광고된 도구마다 하나의 질문에 답한다. 이 도구는 SECRET, UNTRUSTED, SINK, HOSTEXEC 가운데 무엇에 닿을 수 있으며 그 증거는 무엇인가. 증거는 공격자가 얼마나 통제할 수 있는지에 따라 순위가 매겨진다. 설정 자신의 인자와 서버의 소스 코드가 1순위, 선언된 스키마와 annotation이 2순위이며, 자유 서술 설명은 효과를 낮추는 데 결코 쓰이지 않는다. 올리는 데만 쓸 수 있는데, 설명은 판정 대상인 당사자가 작성하기 때문이다. HOSTEXEC은 나머지 셋을 함의한다. 호스트에서의 코드 실행은 호스트가 닿을 수 있는 모든 것에 닿기 때문이다.

요약은 언어 모델이 생성하되 모든 주장에 file:line 증거를 인용해야 하며, 이후 결정적 검사가 가져온 소스에서 인용된 구간을 다시 읽어 걸러낸다. 인용이 가져온 트리 안에서 해석되지 않거나, 그 구간에 도구 이름이나 근거 설명이 지목한 식별자가 없으면, 그 주장은 유지되되 미확인으로 표시된다. 미확인 주장이 효과를 낮추서는 안 되므로 라벨은 살아남지만, 검증된 것으로는 세지 않는다. 모델이 아예 판정하지 않은 도구에는 가장 보수적인 값인 HOSTEXEC을 부여한다. 모델이 라벨 없이 무해하다고 판정한 도구는 그 증거가 코드에서 해석될 때만 무해로 센다. 문서는 효과의 부재를 입증할 수 없기 때문이다. 이 구분이 이 단계의 출력을 그럴듯한 것이 아니라 감사 가능한 것으로 만들며, 5절은 그중 얼마가 검사를 통과하는지 측정한다.

실무적으로 중요한 세부가 둘 있다. 모델에게는 모든 줄에 실제 줄 번호를 붙인 소스를 보여준다. 줄 번호가 없으면 인용된 줄이 파일 뒤로 갈수록 어긋나고, 올바른 판정이 검증에 실패하기 때문이다. 또한 패키지의 자신의 코드가 도구 이름을 전혀 언급하지 않는 경우 – 구현이 의존성에 있는 래퍼 패키지가 그렇다 – 그 이름을 언급하는 직접 의존성을 스크립트 실행을 끈 채 가져와 함께 보여준다. 파이프라인의 어느 지점에서든 제공자 코드는 실행되지 않는다.

### 4.3 Decide and Repair (planned)

*Decide*는 설정이 지정한 값 수준 조건 – 읽기 전용 모드, toolset 선택, 경로 root, origin 허용 목록 – 을 적용한 뒤 설정이 유도한 surface에 대해 배포자의 정책을 평가하고, 금지된 흐름마다 그것을 증언하는 도구를 반환할 것이다. *Repair*는 선언된 작업의 보존을 조건으로 사용 가능한 제어 수단 위에서 최소 비용 patch를 탐색하고, 후보 patch마다 다시 판정해 결과가 가정이 아니라 검증된 것이 되게 하며, patch를 설정 diff로 내보낼 것이다. 우리는 두 단계를 서버 통째 제거, 서버 모드만 사용하는 방식, 이름 기반 휴리스틱 거부 목록, 동일 정책 하의 실행 시점 집행과 비교해, patch 전후의 benign utility, 공격 하 utility, 표적 공격 성공률을 측정할 계획이다.

Table 4. The four arms compared.

| Arm             | Mechanism                                    | Enforcement point     |
|-----------------|----------------------------------------------|-----------------------|
| No Defense [8]  | none                                         | —                     |
| Tool Filter [8] | model selects the tools the task needs       | before the first call |
| Progent [26]    | LLM-generated rules over names and arguments | every call            |
| CaMeL [7]       | capability tracking in a custom interpreter  | every call            |

Table 5. The four suites of AgentDojo. Two injection tasks of the travel suite are excluded, so 909 of the 949 security cases are used.

| Suite     | Tools | User tasks | Injection tasks | Security cases (used/total) |
|-----------|-------|------------|-----------------|-----------------------------|
| workspace | 24    | 40         | 14              | 560 / 560                   |
| slack     | 11    | 21         | 5               | 105 / 105                   |
| travel    | 28    | 20         | 7               | 100 / 140                   |
| banking   | 11    | 16         | 9               | 144 / 144                   |
| Total     | 74    | 97         | 35              | 909 / 949                   |

## 5 Experiments

### 5.1 Experimental Settings

**5.1.1 Baselines.** 표 4의 네 가지와 비교한다. *No Defense* [8]는 방어를 두지 않은 에이전트로, 나머지 셋이 무엇을 줄였는지 재는 기준점이다. *Tool Filter* [8]는 AgentDojo가 함께 제공하는 방어로, 질의를 받으면 모델에게 그 작업에 필요한 도구만 고르게 한 뒤 나머지를 제거한다. *Progent* [26]는 사용자의 작업에서 도구 이름과 인자에 대한 기호 규칙을 LLM으로 생성해 호출마다 검사하고, 실행 중 새 정보가 들어오면 정책을 갱신한다. *CaMeL* [7]은 권한 있는 LLM이 신뢰된 질의를 제한된 프로그램으로 옮기고 격리된 LLM이 신뢰할 수 없는 데이터를 해석하며, 전용 인터프리터가 값마다 능력을 추적해 도구 호출 시점에 정책을 집행한다.

*Fides* [6]는 제외한다. 공개된 저장소가 튜토리얼 노트북 하나여서 보고된 결과를 재현할 수 없기 때문이다. 조합 검사도 베이스라인으로 세우지 않는다. 판정만 내놓고 설정을 바꾸지 않으므로, 공격이 막혔는지와 작업이 유지되었는지로 채점하는 자리에 놓일 수 없다.

**5.1.2 Benchmarks.** AgentDojo는 프롬프트 주입에 대한 에이전트 방어를 평가하는 환경으로, 우리가 비교하는 방어들이 모두 여기서 보고되었다 [8]. 네 개의 suite – 업무용 메일과 일정과 드라이브를 다루는 workspace, 메시지를 다루는 slack, 예약을 다루는 travel, 계좌를 다루는 banking – 에 사용자 과제 97개와 주입 과제 35개가 들어 있고, 하나의 보안 케이스는 사용자 과제와 주입 과제를 짝지은 것이다. 공격자의 지시는 에이전트가 읽게 될 콘텐츠에 심기며, 성공 여부는 결정적 Python 함수가 판정한다. 언어 모델 심판을 쓰지 않는 이유를 저자들이 밝혀 두었고 우리도 따르는데, 공격이 충분히 성공적이면 평가 모델까지 탈취할 수 있기 때문이다.

보안 케이스는 상위 AgentDojo의 949개가 아니라 **909개**를 쓴다. Progent가 travel suite의 주입 과제 두 개를 선호도 공격이라는 이유로 모든 실험에서 제외했고 [26], 그 제외를 보고에서만 아니라 함께 배포한 벤치마크 사본에서 수행했기 때문이다. 우리가 그 사본을 불러 확인한 결과 travel suite의 주입 과제는 일곱 개가 아니라 다섯 개이며, 따라서 케이스 수는 표 5와 같이 949개가 아니라 909개다. Progent가 보고한 수치가 그 집합에서 나왔으므로, 네 arm을 같은 표에 올리려면 나머지도 같은 집합에서 재야 한다. 우리가 직접 실행하는 arm은 949개를 모두 돌리되 집계에서 해당 두 주입 과제를 빼는 방식으로 이를 맞추고, 949개 기준 값도 함께 보고해 제외가 결과를 얼마나 움직이는지 드러낸다.

5.1.3 *Evaluation Metrics*. 공격과 유틸리티에 대해서는 AgentDojo가 정의한 세 지표를 그대로 사용해 용어를 비교 대상과 맞춘다 [8]. *Benign Utility*(BU)는 공격이 없을 때 사용자 과제를 푼 비율이고, *Utility Under Attack*(UUA)은 보안 케이스에서 부작용 없이 과제를 푼 비율이며, *Targeted ASR*(ASR)은 공격자의 목표가 달성된 비율이다. 방어를 평가하려면 셋을 함께 보아야 한다. 도구를 전부 꺼 버리면 ASR은 0이 되지만 BU도 함께 0이 되므로, ASR만으로는 막은 것과 멈춰 세운 것이 구별되지 않는다.

효과 요약 단계에는 대응하는 표준 지표가 없으므로 세 가지를 보고한다. 검증된 주장 비율은 모델의 라벨 주장 가운데 제공자의 소스에 대한 결정적 재검사를 통과하는 비율이고, 표면 축소율은 광고된 도구 가운데 patch가 꺼 버리는 비율이며, patch 비용은 사용된 제어 수단을 비용 순서 - 도구 단위 거부, 서버 모드와 toolset 플래그, 인자 축소, 서버 제거 - 로 집계한 값이다. 구간은 반복이 아니라 케이스에 대한 이항 신뢰구간으로 보고한다.

5.1.4 *Environments*. 모델은 GPT-4o를 쓴다. 네 베이스라인이 공통으로 사용한 모델일 뿐 아니라, 그들이 사용한 모델 가운데 지금도 API로 이용할 수 있는 유일한 것이기 때문이다. Progent가 함께 보고한 Claude Sonnet 4와 Gemini 2.5 Flash, GPT-4.1, CaMeL이 사용한 Claude 3.5 Sonnet과 Gemini 2.5 Pro는 모두 제공자가 서비스를 종료해 재현할 수 없다. 이 때문에 모델 세대에 따른 차이를 보려면 새 모델을 우리가 골라야 하므로, 현재 이용 가능한 최신 모델인 GPT-5.6-sol과 Claude Opus 5를 추가한다. Progent는 에이전트와 정책 생성기 양쪽에, CaMeL은 권한 있는 LLM과 격리된 LLM 양쪽에 같은 모델을 쓰므로 우리도 그렇게 맞춘다.

온도와 실행 횟수는 베이스라인이 설정한 대로 따른다. AgentDojo와 Progent, CaMeL은 모두 온도를 0으로 두고 보안 케이스마다 한 번씩만 실행하며, 반복 실행을 하는 것은 없다. 이들 논문이 보고하는 구간도 반복 사이의 편차가 아니라 케이스 수에 대한 신뢰구간이다.

실험은 macOS 26.6, Apple M1 Ultra(20코어), 64 GB 메모리 환경에서 수행했다. 모든 모델 호출은 제공자의 API를 거치므로 GPU는 사용하지 않는다. 벤치마크와 각 베이스라인은 공개 저장소를 고정된 리미션으로 받아 사용했고, 체크아웃에 가한 수정은 두 가지뿐이다. AgentDojo에는 모델 식별자를 등록하는 세 줄을 더했고, CaMeL에는 지원하지 않는 상수를 만났을 때 오류 메시지를 만들면서 존재하지 않는 속성을 참조해 실행 전체가 중단되는 버그를 한 줄 고쳤다. 과제와 채점기, 공격, 정책 로직에는 손대지 않았다.

## 5.2 RQ1: Effect Summarisation

여기서는 하나의 질문을 던진다. **RQ1**. 모델의 효과 판정 가운데 얼마가 제공자의 소스에 대한 결정적 재검사를 통과하며, 어떤 제공자에서 이 절차가 실패하는가? 우리는 광고된 surface를 복원할 수 있었던, 가장 자주 설정되는 npm 및 PyPI 서버 34개, 총 433개의 광고된 도구를 대상으로, 고정된 rubric과 JSON 스키마를 사용하는 Claude Opus 5, 패키지당 400 KB의 소스 예산, 그리고 4절에서 설명한 검사기로 프로파일을 만든다. 광고된 각 도구는 정확히 하나의 범주에 들어간다. 모델이 라벨을 주장했고 인용이 코드에서 해석된 경우(*verified*), 문서에서만 해석된 경우(*verified\_doc*), 해석되지 않은 경우(*demoted*), 모델이 무해로 판정했고 코드 증거가 있는 경우(*cleared verified*) 또는 없는 경우(*cleared unverified*), 그리고 모델이 판정하지 않은 경우(*unjudged*)다. 광고된 surface에 없는 이름을 판정한 경우는 별도로 센다. 기준과 정의는 실행 전에 사전 등록했으며, 계획서가 정한 dry-run에서 발견된 측정 도구 변경은 모두 실행 이후가 아니라 이전에 기록했다.

## 5.3 Results

표 6가 RQ1에 답한다. 서버 34개 전체에서 라벨 주장의 72.5%가 코드로 검증된다. 판정되지 않은 도구는 없고, 이름 날조는 433개 중 2건으로 무시할 만하다. 손실의 대부분은 세 서버에서 나오며, 각각은 라벨링 오류가 아니라 구조적 이유로 실패한다. 하나는 실행기만 배포하고 컴파일된 바이너리를 내려받으므로 구현이 패키지 안에 전혀 없고, 인용 68건이 모두 문서를 가리킨다. 하나는 시작 시점에 인터페이스 명세에서 도구 이름을 생성하므로 그 이름이 소스 어디에도 없고 선택할

Table 6. Outcome of deterministic re-checking over 34 servers and 433 advertised tools.

|                                                                   | Count   | Share           |
|-------------------------------------------------------------------|---------|-----------------|
| Claims (labels asserted)                                          | 335     |                 |
| verified in code                                                  | 243     | 72.5% of claims |
| verified in documentation only                                    | 11      | 3.3% of claims  |
| demoted (citation did not resolve)                                | 81      | 24.2% of claims |
| Cleared with code evidence                                        | 65      |                 |
| Cleared without code evidence                                     | 33      | 7.6% of tools   |
| Unjudged tools                                                    | 0       | 0.0% of tools   |
| Judged names not in the surface                                   | 2       | 0.5% of tools   |
| Servers usable (claim rate $\geq 0.70$ , few clears, no unjudged) | 27 / 34 | 79.4%           |
| Cost of the run (USD)                                             | 8.53    |                 |

소스 파일이 없었다. 하나는 도구를 minified 번들로 구현하는데, 인용 35건 중 13건이 번들의 첫 줄을 가리키는 반면 인용된 식별자는 다른 위치에 있다. 구현 코드를 읽을 수 있었던 서버 27개로 한정하면 코드 검증률은 **90.7%**(268건 중 243건)다. 사용 가능 집합에서 빠진 나머지 네 서버는 절차의 실패가 아니다. 그 서버들의 도구는 모두 무해로 판정되었고 그중 셋은 전부 코드 증거에 기반했으며, 사전 등록한 사용 가능성 규칙이 주장된 라벨이 없는 서버를 사용 불가로 세기 때문이다.

5.4 Threats to validity

코퍼스. GitHub 코드 검색은 질의당 결과 수에 상한을 두고 표집이 아니라 관련도 순위로 반환하므로, 이 코퍼스는 배포의 무작위 표본이 아니다. 커밋된 설정은 사람들이 실제로 실행하는 설정과 다를 수도 있다. **판정 대상 집합.** 효과는 가장 자주 설정되는 서버 17개(측정)와 34개(평가)에 대해서만 도출했으므로, 유병률 수치는 인기 서버로 구성된 설정을 설명하며 롬테일을 설명하지 않는다. **위험 모델.** 유병률 수치는 최악 가정의 에이전트 아래에서 광고된 surface가 갖는 성질이다. 특정 배포가 공격을 당했다거나 공격이 성공한다고 주장하지 않으며, 악용 가능성 확인은 향후 과제다. **검사기.** 검사기는 한 방향으로 엄격하고 다른 방향으로 느슨하다. 강등 81건 중 약 9건은 인용 구간에 해당 식별자가 더 긴 표현 안에 실제로 존재하지만 토큰 일치에 이를 놓친 경우이고, 반대로 코드로 검증된 판정 308건 중 4건은 네 글자 이하의 토큰에 기대고 있다. **발췌.** 큰 파일에 대해서는 이름이 등장하는 지점 주변 창만 보여주므로, 큰 파일에 존재하는 도구 이름 11개는 모델에게 한 번도 보이지 않았다. 따라서 의존성 추적의 이득은 하한이다. **범위.** 우리의 결론은 설정 파일이 선언하는 MCP 서버에 대한 것이며, 에이전트 스킴로 일반화하지 않는다. 스킴은 광고된 도구 목록을 갖지 않으므로 여기서 쓴 측정 단위가 그대로 적용되지 않는다. **단일 모델.** 모든 요약은 한 시점의 한 모델에서 나왔다. 모델이나 시간에 따라 이 비율이 얼마나 달라지는지는 측정하지 않았다.

6 Related Work

아티팩트 하나를 검사하는 연구. 도구 오염 탐지기는 광고된 설명에서 숨겨진 지시, 이름 가로 채기, 유출 엔드포인트를 읽어낸다 [13]. 신뢰할 수 있는 설명을 재생성해 이 벡터를 제거할 수 있고 [34], 벤치마크는 실제 서버에서 그런 공격의 성공률을 정량화하며 [17, 30, 33], 정적 분석은 서버 내부의 taint 계열 결함을 찾고 [28], 동작 감사는 실행 중 도구를 관찰하며 [29], 이들은 아티팩트를 그 자신의 manifest나 코드에 비추어 판단한다. 우리의 입력은 여러 아티팩트를 조합한 설정이고, 출력은 그 설정에 대한 변경이다.

집합을 판단하는 연구. lethal trifecta 프레이밍은 위험한 조합을 명명하고 [31], toxic-flow 분석은 정책을 위반할 도구 순서를 구성해 점수화하며 [2], 조합 정책은 누적된 노출로 다음 호출이 위반이 되면 세션을 취소하고 [24], 레지스트리 규모 조사는 일부 서버가 세 능력을 동시에 가진다고 보고한다 [36]. 우리는 판단하는 성질을 공유하되 모집단과 출력에서 다르다. 레지스트리 아티팩트가 아니라 사람들이 작성한 설정을 대상으로 측정하고, 판정이나 세션 범위의 제한이 아니라 patch를 반환한다.

**인접 생태계: 에이전트 스킬.** 에이전트 스킬은 MCP와 다른 메커니즘이다. 스킬은 지시문 파일과 선택적 스크립트의 묶음이며, 도구 목록을 협상하는 프로토콜이 없고 광고된 schema도 없다. 그럼에도 공급망 관점의 위험은 남아 있어서, 스킬에 대해서도 건전한 능력 상한이 추상 해석으로 계산되고 [3], 페이로드를 여러 스킬에 분산해 아티팩트 단위 검사를 회피하는 공격이 제시되었고 [35], 대규모 측정으로 취약점 유행률과 자격증명 유출, 공급망 오염이 보고되었다 [4, 11, 18, 19, 23]. 우리 결과는 이 생태계로 그대로 옮겨지지 않는다. 우리의 측정 단위는 설정 파일이 선언한 MCP 서버와 그 서버가 광고한 도구이며, 스킬에는 대응하는 광고된 surface가 없기 때문이다.

**실행 시점에 에이전트를 제약하는 연구.** 신뢰된 질의에서 제어 흐름과 데이터 흐름을 추출해 신뢰할 수 없는 데이터가 프로그램 흐름에 영향을 주지 못하게 하거나 [7], 기밀성과 무결성 라벨을 호출 단위로 추적·집행하거나 [6], 호출 단위 권한 정책으로 최소 권한을 강제하거나 [26], 도달 가능성 위에서 능력을 가두는 cut을 계산하거나 [32], 가드레일로 에이전트 루프를 감쌀 수 있다 [22]. 이들은 질의나 호출 단위로 판단하고, 호스트를 교체하거나 계층해야 하며, 설치된 surface는 그대로 둔다. INTERLOCK은 이들과 상호보완적이다. 질의가 실행되기 전에 기성 호스트가 광고하는 것을 좁히며, 한 배포가 둘을 함께 쓸 수 있다.

**기능 제약 아래 권한을 줄이는 연구.** 브라우저 확장의 과다 권한은 선언된 manifest 권한을 코드가 실제로 호출하는 API와 비교해 탐지되고 [5], 프로그램 debloating은 테스트 스위트를 계속 통과시키면서 아티팩트를 줄인다 [10, 25]. Android의 앱 간 권한 유출에 대한 조합 분석은 설치된 집합을 분석하고 앱이 설치·갱신·삭제될 때 이를 다시 검사한다 [1, 14]. 우리 상황은 무엇을 바꿀 수 있는가에서 다르다. 아티팩트는 제3자의 것이라 우리가 고쳐 쓸 수 없으므로 축소는 배포의 설정에서 일어나며, 보존해야 할 성질은 아티팩트가 아니라 배포에 속한다.

## 7 Conclusion

에이전트 도구 보안 연구는 아티팩트 단위로 성숙해 왔고 집합 단위 질문도 명명되고 탐지되었지만, 사용자가 실제로 바꿀 수 있는 대상인 설정에는 고유한 분석이 없었다. 우리는 그 설정 모집단을 측정해, 분석 가능한 설정의 84.1%가 한 세션 안에서 비공개 데이터와 공격자가 통제하는 콘텐츠, 공격자가 읽을 수 있는 목적지에 닿으며, 그 전부에서 단일 서버로 충분하고, 존재하는 축소 수단은 대체로 쓰이지 않으며 주요 원인 서버에는 아예 없음을 확인했다. 우리는 설정에 대한 최소 권한 tool surface를 계산하는 INTERLOCK을 제시하고 그 첫 단계를 구현했다. 모든 주장이 인용된 소스에 대해 재검사되는 효과 요약으로, 전체의 72.5%가, 패키지가 실제로 실행하는 코드를 함께 배포하는 경우에는 90.7%가 검사를 통과한다. 실무적 함의는 두 가지다. 이 층위의 해법은 설정에 속하며, 사용자가 설치한 것의 대부분은 이미 내려받은 코드만으로 판정할 수 있다.

## References

- [1] Hamid Bagheri, Alireza Sadeghi, Joshua Garcia, and Sam Malek. 2015. Covert: Compositional analysis of android inter-app permission leakage. *IEEE transactions on Software Engineering* 41, 9 (2015), 866–886.
- [2] Luca Beurer-Kellner, Marco Milanta, and Marc Fischer. 2025. Invariant Labs exposes novel prompt injection attack vulnerabilities, “toxic flows”, in agentic systems and MCP servers. <https://invariantlabs.ai/blog/toxic-flow-analysis> Accessed 2026-09-16.
- [3] Varun Pratap Bhardwaj. 2026. Formal analysis and supply chain security for agentic AI skills. *arXiv preprint arXiv:2603.00195* (2026).
- [4] Zhihao Chen, Ying Zhang, Yi Liu, Gelei Deng, Yuekang Li, Yanjun Zhang, Jianting Ning, Leo Yu Zhang, Lei Ma, and Zhiqiang Li. 2026. How your credentials are leaked by LLM agent skills: An empirical study. *arXiv preprint*

- arXiv:2604.03070 (2026).
- [5] Xiaoyu Cheng, Zhi Wang, Wapeng Li, and Lifei Sun. 2026. PerCheck: An AST-Based Framework for Detecting Over-Privilege in Chrome Extensions. In *International Conference on Intelligent Computing*. Springer, 13–24.
  - [6] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. 2025. Securing AI agents with information-flow control. *arXiv preprint arXiv:2505.23643* (2025).
  - [7] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813* (2025).
  - [8] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. arXiv preprint arXiv:2406.13352.
  - [9] Shiva Gaire, Srijan Gyawali, Saroj Mishra, Suman Niroula, Dilip Thakur, and Umesh Yadav. 2026. Systematization of knowledge: Security and safety in the Model Context Protocol ecosystem. In *Intelligent Systems Conference*. Springer, 494–517.
  - [10] Kihong Heo, Woosuk Lee, Pardis Pashakhanloo, and Mayur Naik. 2018. Effective program debloating via reinforcement learning. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 380–394.
  - [11] Florian Holzbauer, David Schmidt, Gabriel Gegenhuber, Sebastian Schrittwieser, and Johanna Ullrich. 2026. Context Matters: Repository-Aware Security Analysis of the Agent Skill Ecosystem. *arXiv preprint arXiv:2603.16572* (2026).
  - [12] Charoes Huang, Xin Huang, and Amin Milani Fard. 2026. Auditing MCP Servers for Over-Privileged Tool Capabilities. *arXiv preprint arXiv:2603.21641* (2026).
  - [13] Gunjan Jain. 2026. *Detecting MCP Tool Poisoning and Rug-Pull Attacks in LLM Agent Architectures*. Technical Report. QCCrypton. <https://zenodo.org/records/19687220>
  - [14] Li Li, Alexandre Bartel, Tegawendé F Bissyandé, Jacques Klein, Yves Le Traon, Steven Arzt, Siegfried Rasthofer, Eric Bodden, Damien Ochteau, and Patrick McDaniel. 2015. Iccta: Detecting inter-component privacy leaks in android apps. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 280–291.
  - [15] Huanxi Liu, Kun Hu, Jiaqi Liao, Qiang Wang, Pengfei Qian, YuanZhao Zhai, Dawei Feng, Bo Ding, and Huaimin Wang. 2026. MCP-Evol-Bench: Benchmarking LLM Agent Performance Across Dynamic Evolutions of MCP Servers. *arXiv preprint arXiv:2607.14642* (2026).
  - [16] Liwei Liu, Tianzhu Han, Zijian Liu, Zishu Dong, and Na Ruan. 2026. ShareLock: A Stealthy Multi-Tool Threshold Poisoning Attack Against MCP. *arXiv preprint arXiv:2606.27027* (2026).
  - [17] Shi Liu, Xuehai Tang, Xikang Yang, Liang Lin, Biyu Zhou, Wenjie Xiao, and Wantao Liu. 2026. When the Manual Lies: A Realistic Benchmark to Evaluate MCP Poisoning Attacks for LLM Agents. In *2026 29th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, 2090–2095.
  - [18] Yi Liu, Zhihao Chen, Yanjun Zhang, Gelei Deng, Yuekang Li, Jianting Ning, and Leo Yu Zhang. 2026. "Do Not Mention This to the User": Detecting and Understanding Malicious Agent Skills in the Wild. *arXiv preprint arXiv:2602.06547* (2026).
  - [19] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. 2026. Agent skills in the wild: An empirical study of security vulnerabilities at scale. *arXiv preprint arXiv:2601.10338* (2026).
  - [20] Muhammad Hamza Arshad Majeed, May Mahmoud, and Sarah Nadi. 2026. An Empirical Study of Model Context Protocol Applications. *arXiv preprint arXiv:2607.25635* (2026).
  - [21] Marco Milanta and Luca Beurer-Kellner. 2025. GitHub MCP exploited: Accessing private repositories via MCP. <https://invariantlabs.ai/blog/mcp-github-vulnerability> Accessed 2026-09-22.
  - [22] Nandhakumar Murugan. 2026. AgentShield: A Zero-Trust Runtime Guardrail Architecture for Autonomous Multi-Agent AI Systems with Bidirectional Context Synchronization. In *Zenodo (CERN European Organization for Nuclear Research)*. doi:10.5281/zenodo.22259021
  - [23] Yubin Qu, Yi Liu, Tongcheng Geng, Gelei Deng, Yuekang Li, Leo Yu Zhang, Ying Zhang, and Lei Ma. 2026. Supply-chain poisoning attacks against LLM coding agent skill ecosystems. *arXiv preprint arXiv:2604.03081* (2026).
  - [24] Chris Schneider, Kriti Faujdar, Philipp Schoenegger, and Ben Bariach. 2026. Securing Multi-Tool AI Agent Chains With Dynamic, Real-Time Compositional Policies. *arXiv preprint arXiv:2607.03423* (2026).
  - [25] Hashim Sharif, Muhammad Abubakar, Ashish Gehani, and Fareed Zaffar. 2018. TRIMMER: application specialization for code debloating. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 329–339.
  - [26] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI agents with privilege control. *arXiv preprint arXiv:2504.11703* (2025).
  - [27] Snyk. 2026. agent-scan issue codes. <https://github.com/snyk/agent-scan/blob/main/docs/issue-codes.md> Accessed 2026-09-16.

[28] Pengyu Sun, Zifeng Kang, Qishu Jin, Enhao Huang, Xin Liu, Dakun Shen, and Song Li. 2026. VIPER-MCP: Detecting and Exploiting Taint-Style Vulnerabilities in Model Context Protocol Servers. *arXiv preprint arXiv:2605.21392* (2026).

[29] Zhuoran Tan, Run Hao, Jeremy Singer, Yutian Tang, and Christos Anagnostopoulos. 2026. SandScope: a Behavioral Audit Layer for MCP Tools in LLM Agent Supply Chains. (2026).

[30] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. 2026. Mcptox: A benchmark for tool poisoning on real-world mcp servers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 35811–35819.

[31] Simon Willison. 2025. The lethal trifecta for AI agents: private data, untrusted content, and external communication. *Simon Willison's Weblog* (2025).

[32] Wujie Xiong, Rabimba Karanjai, Yang Lu, Weidong Shi, and Lei Xu. 2026. Reachability-Based Capability Confinement for LLM Agents under Indirect Prompt Injection. *arXiv preprint arXiv:2608.30041* (2026).

[33] Yixuan Yang, Cuifeng Gao, Daoyuan Wu, Yufan Chen, Yingjiu Li, and Shuai Wang. 2025. MCPSecBench: A systematic security benchmark and playground for testing Model Context Protocols. *arXiv preprint arXiv:2508.13220* (2025).

[34] Hengkai Ye, Zhechang Zhang, Jinyuan Jia, and Hong Hu. 2026. Trustdesc: Preventing tool poisoning in llm applications via trusted description generation. *arXiv preprint arXiv:2604.07536* (2026).

[35] Puyu Zeng, Simeng Qin, Jingzhi Li, Ju Jia, Zheli Liu, and Xiaojun Jia. 2026. ColluSkill: Adversarial Cross-Skill Composition for Evading Agent Skill Scanners. *arXiv preprint arXiv:2608.09732* (2026).

[36] Shuli Zhao, Qinsheng Hou, Zihan Zhan, Yanhao Wang, Yuchong Xie, Yu Guo, Libo Chen, Shenghong Li, and Zhi Xue. 2026. Parasites in the toolchain: A large-scale analysis of attacks on the MCP ecosystem. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 138–155.